Sheaf Cohomology for Settlement Protocol Verification

Measuring Topological Completeness and Information Asymmetry in ACTP

Justin Rooschüz* Damir Mujić†

AGIRAILS Inc.

April 29, 2026

Abstract

Autonomous AI agents are non-deterministic: they can refuse to deliver, refuse to pay, or interpret the same task differently each time. This makes agent-to-agent commerce structurally risky, and the classical answer is escrow — a neutral party that holds funds until both sides perform. For autonomous agents, that neutral party is a smart contract. But this only relocates the problem. The escrow protocol itself must be provably correct, and "provably correct" is not one thing.

This paper asks a specific part of that question: when an agent settlement protocol exposes local state at each step, does that state assemble into a globally consistent picture, and if not, where does it fail? We answer with cellular sheaf cohomology. Two sheaves are constructed over the 8-state lifecycle of ACTP, the Agent Commerce Transaction Protocol. A state-based sheaf measures structural completeness: whether local state globalises. A participant-based sheaf measures information asymmetry: whether all parties can see what they need to verify the outcome.

On the state sheaf, $H^1 = 24$ when the protocol is treated as a bare graph and $H^1 = 0$ once 2-cells encoding its parallel paths are added. The protocol is topologically complete: every piece of local state carries through every transition into a single consistent global view. On the participant sheaf, under a conservative semantic model, H^1 stays between 6 and 8 at every protocol state. That gap is not an error. It is the information that still needs to be exchanged off-chain or via an oracle for all parties to agree — and quantifying it is part of the contribution. A separate exact six-channel raw-visibility supplement yields $H^1 = 0$ at every state after public-face completion, so the conservative semantic values are best read as an upper-bound diagnostic rather than an exact raw-visibility invariant.

Structural completeness is necessary but not sufficient for trustlessness. The two sheaves answer orthogonal questions and are meant to be read together. We also provide open-source tooling: a YAML-driven H^1 linter with CI integration; 1-complex rank computations cross-validated using exact rational arithmetic; and a participant visibility model checked against the live ACTP SDK as of 2026. To our knowledge this is the first application of sheaf cohomology to smart contract escrow protocol verification, building on Felber et al.'s task-sheaf framework but extending it with a participant information-asymmetry sheaf.

*justin@agirails.io

†damir@agirails.io

Contents

1 Introduction **3**

2 Background **5**
- 2.1 Cellular Sheaves . . . 5
- 2.2 The Agent Commerce Transaction Protocol (ACTP) . . . 6

3 State-Based Sheaf Construction **7**
- 3.1 Vertex and Edge Stalks . . . 7
- 3.2 Coboundary Matrices . . . 8
- 3.3 Results: 1-Complex . . . 9
- 3.4 2-Cell Extension . . . 10

4 Dispute Process and Attestation **11**
- 4.1 Current Dispute Process . . . 11
- 4.2 Dispute-Flow Interpretation . . . 11
- 4.3 Attestation Layer . . . 11

5 Participant-Based Sheaf **12**
- 5.1 Construction . . . 12
- 5.2 Results . . . 13
- 5.3 Implications for Trustlessness . . . 14

6 Tooling **15**
- 6.1 Protocol Specification Format . . . 15
- 6.2 h1_engine.py: Core Computation . . . 16
- 6.3 h1_lint.py: CI Integration . . . 16

7 Related Work **17**

8 Discussion **17**

9 Conclusion **20**

A ACTP v2.7 Transition Table **22**

B Public 2-Cell Decomposition **22**

C Mainnet Execution Trace **22**

D Participant Semantic Supplement **23**

E Participant Higher-Order Supplement **23**

F Euler Characteristic Cross-Check **24**

1 Introduction

Autonomous AI agents are non-deterministic. Given the same task twice, an agent may produce different outputs, take different paths to reach them, or decide not to proceed at all. This is not a deficiency. It is a consequence of the decision-making capacity that makes agents useful in the first place. A deterministic agent is a program. A non-deterministic agent is an entity that chooses.

But non-determinism makes economic interaction between agents structurally risky. An agent that receives payment before delivering may not deliver. An agent that delivers before being paid may not get paid. The same pattern exists in human commerce, and it is why escrow exists: a neutral third party holds funds until both sides perform. For autonomous agents, that neutral third party is a smart contract.

This does not eliminate the problem. It relocates it. A smart contract is only as trustworthy as its design, and "trustworthy" needs to mean something specific when the contract is supposed to enforce guarantees no human is watching. A bug in the escrow logic, a missing state transition, an asymmetry in what each party can observe at a critical moment — any of these can break the guarantee that the escrow was supposed to provide.

So the question becomes: how do you prove that an escrow protocol actually holds together? Not test it. Not audit the code line by line. *Prove* it, in the sense that a structural property of the protocol — independent of any particular execution or any particular auditor — can be formally verified and independently reproduced.

What existing tools answer, and what they miss. Model checkers like TLA$^+$ [10] and Alloy [9] verify temporal and relational properties by enumerating reachable states and checking that safety and liveness invariants hold across traces. They are powerful, but they treat all data as undifferentiated. They cannot distinguish a missing *dimension* of shared state from a missing *transition*. Smart-contract auditors like Certora [3], Echidna [5], and Slither [12] check code-level safety: reentrancy, overflow, access control. They operate one level below the protocol, on the Solidity implementation rather than the state machine the implementation is meant to realise. Both layers are necessary, and neither addresses the question we want to answer: does the information the protocol exposes at each step actually assemble into a globally consistent picture, and if not, where does it fail?

Sheaf cohomology as the framework. This is the question cellular sheaf cohomology was designed to answer. A cellular sheaf assigns a vector space — the *stalk* — to each cell of a cell complex, and equips adjacent cells with *restriction maps* encoding how data flows across boundaries. The first cohomology group H^1 counts the independent obstructions to globalising locally consistent data. $H^1 = 0$ means every assignment of local data that agrees on shared dimensions can be assembled into a globally consistent section. $H^1 \neq 0$ gives a precise, computable count of the dimensions that prevent this.

Applied to a settlement protocol, this yields two complementary measurements. A *state-based sheaf* places stalks on protocol states (their observable data dimensions) and on transitions (the fields that must agree across them); its H^1 measures structural completeness — whether local state at each step assembles into a consistent global view. A *participant-based sheaf* places stalks on the protocol's principals (requester, provider, escrow vault, kernel) and on the information

channels between them; its H^1 measures information asymmetry — whether any dimension of protocol state is accessible to some principals but hidden from others.

The two sheaves answer orthogonal questions. A protocol can be structurally complete yet participant-asymmetric, with one party unable to verify what another knows. Trustlessness in a strict sense requires both, and the participant sheaf further admits two natural resolutions: a conservative *semantic* model that treats off-chain evidence as opaque, and an exact *raw-visibility* model that credits public on-chain observability through public view functions. The gap between these two models formalises the difference between what participants can *see* and what they can *verify*.

Application: ACTP. The application in this paper is the Agent Commerce Transaction Protocol (ACTP) [1], an 8-state escrow lifecycle for autonomous agents running on Base. ACTP handles request, quote, commitment, in-progress work, delivery, and settlement, with side branches for disputes and cancellations. The protocol targets Base mainnet deployment, and at least one end-to-end settlement transaction has been observed on Base mainnet [2]. The implementation we analyse is `sdk-js` package version `3.2.0` (commit `d5dc886`) and the corresponding `actp-kernel` commit `91f4f90`.

Scope of formal claims. Unless explicitly marked otherwise, every theorem and rank computation in Sections 3 through 5 is a statement about a reduced supplementary artifact — a YAML protocol specification labelled "ACTP v2.7" — and, for 2-complex claims, about an explicit 10-face refinement declared in Appendix B. Code inspection and the mainnet transaction trace are included as implementation evidence. They are not premises in the cohomology proofs. This distinction matters because the paper's mathematical claims are about a model, and the model's fidelity to the deployed protocol is a separate, empirical question that we address but do not conflate with the proofs.

Related work. Felber et al. [6] recently applied cellular sheaves to distributed task solvability, showing that sheaf cohomology encodes obstructions to finding valid protocol solutions. Their framework treats distributed computing tasks in general; ours addresses a specific class of them — settlement protocols — and adds participant information asymmetry as a second construction. Background on cellular sheaves is drawn from Ghrist [7] and Curry [4]; the spectral theory of Hansen and Ghrist [8] provides the algebraic foundation. Robinson [11] situates sheaf theory within signal processing on networks, which informs our restriction-map construction.

Paper roadmap. Section 2 reviews cellular sheaves and ACTP. Section 3 constructs the state-based sheaf and derives H^1 for the 1-complex and 2-complex. Section 4 analyses the dispute process and the attestation layer. Section 5 presents the participant-based sheaf. Section 6 describes the current supplementary tooling. Section 7 situates the work. Section 8 discusses limitations and future work. Section 9 concludes.

2 Background

2.1 Cellular Sheaves

Before the formalism, a one-line intuition. A cellular sheaf is a way to attach local data to the pieces of a space (vertices, edges, faces) and ask: do all the local pieces fit together into a consistent global picture? When they do, the sheaf has a global section. When they do not, the obstruction is counted by cohomology. This section makes that precise.

Let X be a regular cell complex. Write $X^{(k)}$ for the set of k-cells and $\sigma \leq \tau$ when cell σ is a face of cell τ.

Definition 2.1 (Cellular sheaf)**.** A cellular sheaf $\mathcal{F}$ on X over $\mathbb{R}$ consists of

1. a finite-dimensional $\mathbb{R}$-vector space $\mathcal{F}(\sigma)$ for each cell $\sigma \in X$, and
2. for each face relation $\sigma \leq \tau$, a linear map

$$\rho_{\sigma \to \tau} \colon \mathcal{F}(\sigma) \to \mathcal{F}(\tau),$$

subject to functoriality: if $\sigma \leq \tau \leq \upsilon$ then $\rho_{\sigma \to \upsilon} = \rho_{\tau \to \upsilon} \circ \rho_{\sigma \to \tau}$.

Cochain complex. Given $\mathcal{F}$ on a cell complex with 0-cells (vertices) V, 1-cells (edges) E, and 2-cells (faces) F, orient each edge $e = (u, v)$ with tail $t(e) = u$ and head $h(e) = v$, and orient each face f with an ordered boundary ∂f. Define

$$C^0 = \bigoplus_{v \in V} \mathcal{F}(v), \quad C^1 = \bigoplus_{e \in E} \mathcal{F}(e), \quad C^2 = \bigoplus_{f \in F} \mathcal{F}(f).$$

The **coboundary maps** are:

$$(\delta_0 \xi)(e) = \rho_{h(e) \to e}\big(\xi(h(e))\big) - \rho_{t(e) \to e}\big(\xi(t(e))\big), \tag{1}$$

$$(\delta_1 \eta)(f) = \sum_{e \in \partial f} \epsilon(e, f)\, \rho_{e \to f}\big(\eta(e)\big), \tag{2}$$

where $\epsilon(e, f) \in \{+1, -1\}$ is the incidence sign.

Definition 2.2 (Sheaf cohomology)**.** The cohomology groups of $\mathcal{F}$ are

$$H^0(\mathcal{F}) = \ker(\delta_0), \quad H^1(\mathcal{F}) = \frac{\ker(\delta_1)}{\operatorname{im}(\delta_0)}, \quad H^2(\mathcal{F}) = \frac{C^2}{\operatorname{im}(\delta_1)}.$$

Remark 2.3. $H^0(\mathcal{F})$ is the space of *global sections*: assignments of vectors to vertices that are consistent across every edge. Think of it as the set of ways the local data can be stitched together without tearing. $H^1(\mathcal{F})$ counts the independent obstructions to stitching: $\dim H^1 = \dim \ker(\delta_1) - \operatorname{rank}(\delta_0)$. For a 1-complex (no 2-cells), $\ker(\delta_1) = C^1$ so $\dim H^1 = \dim C^1 - \operatorname{rank}(\delta_0)$, which can overcount when cycles carry redundant information. Adding 2-cells reduces H^1 by $\operatorname{rank}(\delta_1)$. Put differently: $H^1 = 0$ means every locally consistent assignment extends to a global one; $H^1 > 0$ means some do not, and the number tells you how many dimensions of disagreement exist.

2.2 The Agent Commerce Transaction Protocol (ACTP)

Before the formal description, a concrete scenario. A requester agent needs a document translated and posts a task with a deadline. A provider agent responds with a price. The requester accepts, and USDC is locked in escrow on-chain. The provider does the work and marks it delivered. If the requester accepts the result, the funds release to the provider. If not, either side can open a dispute inside a fixed window, and resolution follows a privileged path. This is ACTP.

Formally, ACTP is the settlement layer of the AGIRAILS infrastructure [1]. It defines a payment lifecycle in which an AI agent (*Requester*) purchases a service from another agent (*Provider*) via a non-custodial on-chain USDC escrow enforced by the *ACTPKernel* smart contract.

States. ACTP defines eight protocol states:

$$\textsc{Initiated} \to \textsc{Quoted} \to \textsc{Committed} \to \textsc{InProgress} \to \textsc{Delivered} \to \textsc{Settled},$$

with side-branches to Disputed (dispute) and Cancelled (cancellation). Table 1 lists each state and the primary data dimensions observable at that state.

Table 1: ACTP v2.7 reduced observable basis used by the current supplementary artifact.

State	Dim	Observable dimensions (stalk basis)
Initiated	6	transaction_id, requester_address, provider_address, amount, deadline, service_hash
Quoted	7	+ quote_details
Committed	8	+ escrow_id, escrow_balance
InProgress	9	+ work_started_at
Delivered	11	+ delivery_proof, dispute_window_end
Settled	9	transaction_id, requester_address, provider_address, amount, service_hash, escrow_id, settlement_tx, provider_payout, platform_fee
Disputed	10	transaction_id, requester_address, provider_address, amount, deadline, service_hash, escrow_id, escrow_balance, delivery_proof, dispute_reason
Cancelled	6	transaction_id, requester_address, provider_address, amount, cancellation_reason, refund_tx

Remark 2.4 (Semantic observables vs. raw storage)**.** The stalk basis in Table 1 is a basis of *semantic observables*, not a one-to-one copy of raw Solidity storage slots. Depending on the dimension, an observable may be assembled from contract storage, events, calldata, SDK-managed payloads, or public off-chain artifacts. This distinction matters for participant visibility: public hashes or references are not equivalent to full semantic content.

Transitions. The 12 directed transitions of the current reduced-basis ACTP v2.7 supplementary artifact are:

1. INITIATED $\xrightarrow{e_1}$ QUOTED: provider submits quote
2. INITIATED $\xrightarrow{e_2}$ COMMITTED: requester links funded escrow directly
3. INITIATED $\xrightarrow{e_3}$ CANCELLED: requester cancels before quote
4. QUOTED $\xrightarrow{e_4}$ COMMITTED: requester accepts quote and links escrow
5. QUOTED $\xrightarrow{e_5}$ CANCELLED: quote expires or is rejected
6. COMMITTED $\xrightarrow{e_6}$ INPROGRESS: provider begins execution
7. COMMITTED $\xrightarrow{e_7}$ CANCELLED: post-commit cancellation / refund path
8. INPROGRESS $\xrightarrow{e_8}$ DELIVERED: provider marks delivery
9. INPROGRESS $\xrightarrow{e_9}$ CANCELLED: execution abort / refund path
10. DELIVERED $\xrightarrow{e_{10}}$ SETTLED: requester confirms, or anyone settles after the dispute window
11. DELIVERED $\xrightarrow{e_{11}}$ DISPUTED: requester or provider opens dispute within the dispute window
12. DISPUTED $\xrightarrow{e_{12}}$ SETTLED: privileged dispute resolution

The live contract additionally permits a privileged DISPUTED $\to$ CANCELLED resolution edge. The current open-source state-sheaf artifact omits that administrative refund edge, so the numerical state-based results in this paper apply to the 12-edge artifact above unless stated otherwise.

Parallel paths and cycles. ACTP has three structurally significant parallel paths:

- *Direct happy path:* INITIATED $\to$ COMMITTED $\to$ INPROGRESS $\to$ DELIVERED $\to$ SETTLED (4 transitions).
- *Quoted happy path:* INITIATED $\to$ QUOTED $\to$ COMMITTED $\to$ INPROGRESS $\to$ DELIVERED $\to$ SETTLED (5 transitions).
- *Dispute path:* DELIVERED $\to$ DISPUTED $\to$ SETTLED in parallel with DELIVERED $\to$ SETTLED (creates a cycle / face).

Four cancellation branches from INITIATED, QUOTED, COMMITTED, INPROGRESS each form short paths to CANCELLED. In the current public 2-complex artifact, these parallel paths are encoded explicitly as faces in the YAML supplement, while the same linter can still be run on the underlying 1-complex alone.

Figure 1 depicts the ACTP state machine.

3 State-Based Sheaf Construction

3.1 Vertex and Edge Stalks

The state-based sheaf $\mathcal{F}_S$ attaches a vector space to every protocol state and every transition between states.

Definition 3.1 (State stalk)**.** The stalk $\mathcal{F}_S(v)$ at state v is $\mathbb{R}^d$ where $d = \dim(v)$ is the number of named data fields observable at that state (Table 1).

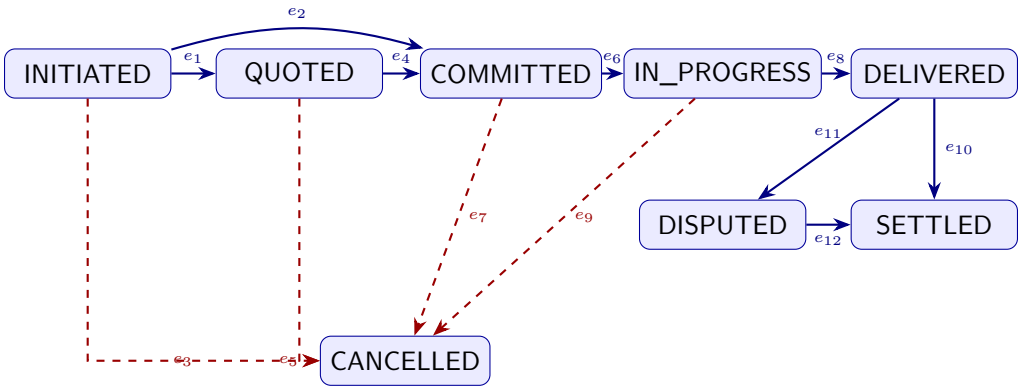


Figure 1: ACTP v2.7 state machine for the 12-edge supplementary artifact. Blue solid arrows = nominal path; red dashed arrows = cancellation. The live contract additionally exposes a privileged Disputed → Cancelled resolution edge not drawn here.

Definition 3.2 (Transition stalk)**.** For transition $e = (u, v)$, the stalk $\mathcal{F}_S(e) = \mathbb{R}^s$ where s is the number of fields *shared* between u and v — i.e., fields that must agree across the transition.

Definition 3.3 (Restriction maps)**.** For an edge $e = (u, v)$:

- $\rho_{u \to e} \colon \mathcal{F}_S(u) \to \mathcal{F}_S(e)$: projection onto the shared fields.
- $\rho_{v \to e} \colon \mathcal{F}_S(v) \to \mathcal{F}_S(e)$: projection onto the shared fields.

The coboundary δ_0 is therefore (from (1)):

$$(\delta_0 \xi)(e) = \rho_{v \to e}(\xi(v)) - \rho_{u \to e}(\xi(u)),$$

which is zero precisely when the data at u and v agree on all shared dimensions.

Figure 2 illustrates the construction on a single ACTP transition, $e_1 \colon$ Initiated → Quoted.

3.2 Coboundary Matrices

Let $n_v = \sum_v \dim(v)$ and $n_e = \sum_e \dim(e)$. For the current reduced-basis ACTP v2.7 supplementary artifact:

$$\dim(C^0) = 6 + 7 + 8 + 9 + 11 + 9 + 10 + 6 = 66,$$
$$\dim(C^1) = \sum_{i=1}^{12} s(e_i)$$

where $s(e_i)$ is the shared-field count for each transition. After computing shared dimensions for each of the 12 transitions (e.g., $s(e_1) = 6$ for Initiated → Quoted, $s(e_{10}) = 6$ for Delivered → Settled), we obtain $\dim(C^1) = 72$.

Remark 3.4. The current YAML-driven engine encodes each state's `stalks` and derives edge overlap by field intersection. It programmatically assembles δ_0 and, when a `faces:` section is present, also assembles δ_1. Running `python3 h1_lint.py --2complex` on the public supplement reproduces the 2-complex results reported below.

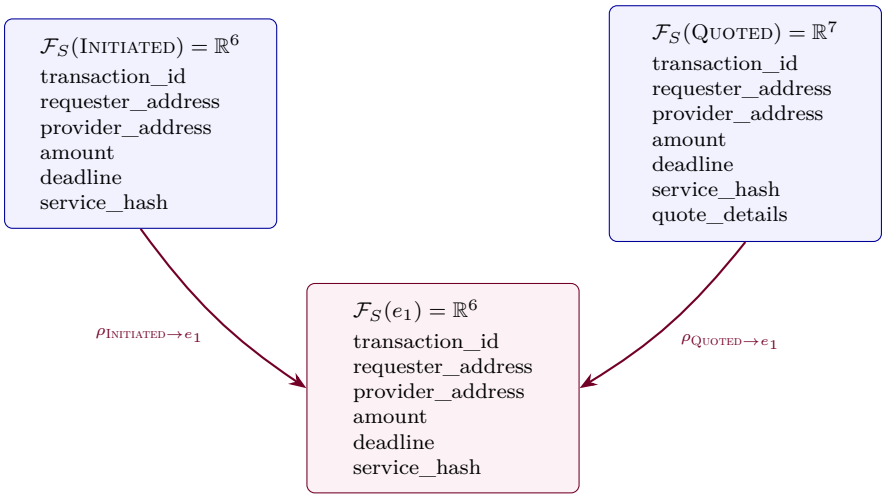


Figure 2: The state-based sheaf over the transition e_1: INITIATED $\to$ QUOTED. Each vertex stalk is a real vector space whose basis is the observable fields at that state. The edge stalk is the subspace spanned by the fields shared between the two endpoints: all six INITIATED fields carry through to QUOTED. The restriction maps are orthogonal projections of each vertex stalk onto the edge stalk. The coboundary $\delta_0\xi(e_1)$ is zero precisely when the restrictions from the two endpoints agree. Here the only field in QUOTED without a counterpart in INITIATED is `quote_details`, which is added by the transition and has no corresponding component in $\mathcal{F}_S(e_1)$.

The matrix $\delta_0 \in \mathbb{R}^{72\times 66}$ has rank computed via NumPy `linalg.matrix_rank` with tolerance 10^{-9}. All rank computations for δ_0 (the 1-complex results) have been cross-validated using exact rational arithmetic (Python `fractions` module, see supplementary `verify_rational.py`); results agree with the floating-point computation in every reported case.

3.3 Results: 1-Complex

Table 2 summarises H^1 for several subgraphs of ACTP v2.7, treating the protocol as a 1-complex (no 2-cells).

Table 2: H^1 on the 1-complex for ACTP v2.7 subgraphs in the current supplementary artifact.

Subgraph	$\dim C^0$	$\dim C^1$	$\text{rank}(\delta_0)$	$\dim H^1$	**Interpretation**
Full protocol	66	72	48	24	Aggregate obstruction count
Direct happy path	43	29	29	0	Direct funded path is consistent
Quoted linear path	50	35	35	0	Quoted route alone is consistent
Quote/commit comparison	50	41	35	6	Quote-specific data not preserved across both commit routes
Dispute path	30	21	15	6	Dispute subgraph asymmetry
Cancellation subgraph	36	42	24	18	Terminal-state field loss dominates

Interpretation. The 18 cancellation obstructions arise because the four cancellation branches from INITIATED, QUOTED, COMMITTED, and INPROGRESS each arrive at CANCELLED carrying only the basic transactional core (`transaction_id`, `requester_address`, `provider_address`, `amount`), leaving terminal-state fields unmatched. The 6 dispute obstructions reflect that DISPUTED

and SETTLED do not share all fields that the direct DELIVERED → SETTLED path would carry. The additional 6 obstructions do not arise on the quoted linear path by itself. They arise when the direct route INITIATED → COMMITTED is compared with the quoted route INITIATED → QUOTED → COMMITTED inside the same 1-complex subgraph.

3.4 2-Cell Extension

The 1-complex count of 24 is an artefact: it treats parallel paths as independent rather than recognising them as boundaries of 2-cells.

Definition 3.5 (Face / 2-cell)**.** A *face* of the state graph is specified by two directed paths p^+ and p^- with the same source and target. Its boundary is the formal difference

$$\partial f = p^+ - p^-,$$

and the face stalk $\mathcal{F}_S(f)$ is the intersection of all edge stalks that appear on ∂f.

Remark 3.6. This convention selects the maximal face stalk compatible with the restriction structure. Other choices (e.g., a strict subspace) exist but do not affect the $H^1 = 0$ result, since any subspace of this maximal stalk still fills the same 1-cycles.

In the current public ACTP artifact, the 10 explicit faces comprise one quote/commit face, one dispute/settle face, and eight cancellation faces. The most significant face is the *dispute face*: the parallel-path pair DELIVERED $\xrightarrow{e_{11}}$ DISPUTED $\xrightarrow{e_{12}}$ SETTLED versus DELIVERED $\xrightarrow{e_{10}}$ SETTLED.

Theorem 3.7 (2-Complex completeness)**.** *Let $\mathcal{F}_S$ be the state-based sheaf for the current ACTP v2.7 supplementary artifact, equipped with the explicit 10-face refinement listed in Appendix B. Then*

$$\dim H^1(\mathcal{F}_S) = 0.$$

Proof sketch. We construct the coboundary map $\delta_1 \colon C^1 \to C^2$ using (2), verify $\delta_1 \circ \delta_0 = 0$ numerically (residual $= 0.0 \times 10^0$ in the public CLI output), and compute

$$\dim H^1 = \dim \ker(\delta_1) - \operatorname{rank}(\delta_0).$$

The public command `python3 h1_lint.py --2complex` returns $\dim C^2 = 44$, $\operatorname{rank}(\delta_1) = 24$, and $\dim \ker(\delta_1) = 48 = \operatorname{rank}(\delta_0)$, hence $H^1 = 0$. □

Remark 3.8. The condition $\dim \ker(\delta_1) = \operatorname{rank}(\delta_0) = 48$ implies *exactness* at C^1: every cocycle is a coboundary. This is the algebraic statement of topological completeness — there is no "phantom" dimension of edge data that closes up around every face but cannot be explained by consistent vertex data.

Corollary 3.9. $\dim H^2(\mathcal{F}_S) = \dim C^2 - \operatorname{rank}(\delta_1) = 44 - 24 = 20$.

In protocol terms, H^2 counts independent "volume" obstructions at the 2-cell level: directions in which three or more parallel paths could disagree even when every pair of paths is locally consistent. A full interpretation is left for future work, but the $H^2 = 20$ count is likely dominated by the cancellation branches. Four routes converge at CANCELLED carrying different field sets, and the cancellation subgraph already contributes 18 of the 24 obstructions on the 1-complex (Table 2); higher-order disagreements between these branches are the natural candidate for what 2-cells absorb but do not resolve.

Table 3 summarises the full dimensional accounting.

Table 3: Cochain dimensions and cohomology for ACTP v2.7 in the current public 2-complex artifact.

	$\dim C^0$	$\dim C^1$	$\dim C^2$
ACTP v2.7	66	72	44
$\operatorname{rank}(\delta_0)$	48		
$\operatorname{rank}(\delta_1)$	24		
$\dim \ker(\delta_1)$	48		
$\dim H^0$	$66 - 48 = 18$		
$\dim H^1$	$48 - 48 = \mathbf{0}$		
$\dim H^2$	$44 - 24 = 20$		

4 Dispute Process and Attestation

4.1 Current Dispute Process

The current ACTP implementation does *not* include a distinct mediation state. The dispute process is the 8-state flow already described above:

$$\textsc{Delivered} \to \textsc{Disputed} \to \textsc{Settled} \quad \text{or} \quad \textsc{Delivered} \to \textsc{Disputed} \to \textsc{Cancelled}.$$

The live contract further allows direct $\textsc{Delivered} \to \textsc{Settled}$ completion, with requester-led settlement inside the dispute window and permissionless settlement after the dispute window expires.

Remark 4.1. Earlier drafts referred to a "mediation" vertex. That was a modelling error rather than a committed protocol roadmap item. The defensible paper model is the implemented dispute process, not a hypothetical mediation state.

4.2 Dispute-Flow Interpretation

The implemented dispute process accounts for the main asymmetry claims in the paper. On the 1-complex, the dispute subgraph contributes $\dim H^1 = 6$: the direct settlement path and the dispute-resolution path do not carry identical information. At the protocol level this matters because dispute resolution is still privileged. The contract lets either side open a dispute, but resolution out of Disputed is not participant-symmetric. The design point worth highlighting is therefore not "missing mediation" but participant asymmetry inside the existing dispute process, which is what the paper measures.

4.3 Attestation Layer

As a paper-level augmentation experiment, we add attestation-related observables to the post-delivery portion of the lifecycle in order to test how new verification metadata changes the obstruction count.

On the 1-complex, this raises $\dim H^1$ from 24 to 26.

Remark 4.2 (Attestation augmentation)**.** In the ACTP attestation experiment, the new observables appear on multiple post-delivery states, widening certain edge intersections on that branch

of the protocol graph. Because those observables are not matched across all parallel routes with the same endpoints, the resulting increase in $\dim C^1$ is not absorbed by a corresponding increase in $\operatorname{rank}(\delta_0)$, yielding the observed change from 24 to 26 on the 1-complex. This is a branch-dependent, ACTP-specific empirical finding, not a general theorem about arbitrary data augmentation.

Remark 4.3. Anchoring an `attestationUID` on-chain does not itself validate EAS semantics at kernel level. Attestation verification is an optional SDK-side step, so attestation is a public verification layer rather than a kernel-enforced settlement condition. Given a UID, the EAS attestation content is also publicly queryable; the participant model in this paper does not separately count that additional shared read path.

5 Participant-Based Sheaf

5.1 Construction

Two participant sheaves. To avoid conflating two distinct questions, we distinguish two participant models throughout this section:

$$\mathcal{F}^s_{P,\text{sem}} \qquad \text{and} \qquad \mathcal{F}^s_{P,\text{raw}}.$$

Here $\mathcal{F}^s_{P,\text{sem}}$ denotes the conservative four-channel semantic upper-bound model used in Table 4, while $\mathcal{F}^s_{P,\text{raw}}$ denotes the exact six-channel raw-visibility complex with explicit public-information faces reported in Appendix E.

The values in Table 4 are therefore $H^1(\mathcal{F}^s_{P,\text{sem}})$, whereas the zero values in Appendix E are $H^1(\mathcal{F}^s_{P,\text{raw}})$. These are not two competing computations of the same sheaf. They answer different sheaf-theoretic questions.

Concretely: at the DELIVERED state, the Requester, the Provider, the EscrowVault and the ACTPKernel can all inspect the 32-byte hash of the delivery proof anchored on-chain. In the raw-visibility sense they share the same public field. But only the Requester and Provider possess the actual content the hash commits to — the document, the evidence, the off-chain payload itself. The hash is shared. The meaning is not. The raw-visibility sheaf $\mathcal{F}^s_{P,\text{raw}}$ registers the hash as a public field visible to everyone; the semantic sheaf $\mathcal{F}^s_{P,\text{sem}}$ treats the hash as opaque unless the principal also holds the content it commits to. This is the distinction the two sheaves make precise.

The semantic participant sheaf $\mathcal{F}^s_{P,\text{sem}}$ is defined on a cell complex whose vertices are the four ACTP principals:

$$V_P = \{\,\text{Requester},\ \text{Provider},\ \text{EscrowVault},\ \text{ACTPKernel}\,\}.$$

Edges model *information channels*: the pairwise connections through which principals share data. In the current implementation, six channels are relevant:

$$\begin{aligned} c_1 &= (\text{Requester}, \text{Provider}), & c_2 &= (\text{Requester}, \text{ACTPKernel}),\\ c_3 &= (\text{Provider}, \text{ACTPKernel}), & c_4 &= (\text{EscrowVault}, \text{ACTPKernel}),\\ c_5 &= (\text{Requester}, \text{EscrowVault}), & c_6 &= (\text{Provider}, \text{EscrowVault}). \end{aligned}$$

Because on-chain kernel and vault state are publicly readable, c_5 and c_6 reduce some asymmetries that were overstated in earlier four-channel drafts. For conservative analysis, however, we still use a simplified four-channel abstraction to obtain upper bounds on participant asymmetry.

Remark 5.1. A naive upgrade from the four-channel participant graph to the full six-edge K_4 graph would change the topology of the participant complex itself. On a graph-only K_4, even fully public observables generate cycle classes coming from the participant graph rather than from genuine visibility asymmetry. Accordingly, an exact six-channel participant computation should be carried out on a higher-order participant complex with explicit public-information faces, not by merely adding two edges to the current graph model.

Definition 5.2 (Participant stalk)**.** At each protocol state s, principal p has stalk $\mathcal{F}^s_{P,\mathrm{sem}}(p) = \mathbb{R}^{k_p}$ where k_p is the number of fields that p can observe at state s under ACTP's public-state and payload-availability rules. The restriction maps are projections onto the fields observable through the corresponding channel.

Remark 5.3. EscrowVault is modelled as a passive principal: its stalk records data accessible through its public view surface and escrow-specific storage, not a claim about active querying or agency.

Stalk dimensions are derived from the Solidity view surface (`getTransaction()`), public vault storage, emitted events, and SDK-managed off-chain payloads. Public on-chain fields are therefore visible to all principals; the residual asymmetry comes primarily from semantic payloads and evidentiary content that are not fully replicated through all channels. The participant visibility model was validated against the ACTP SDK v3.2.0 implementation (`sdk-js`) and the `ACTPKernel.sol` smart contract; see supplementary validation report for details.

5.2 Results

Table 4 reports conservative upper-bound values for $\dim H^1(\mathcal{F}^s_{P,\mathrm{sem}})$ under the simplified four-channel participant model.

Table 4: Participant-sheaf upper-bound $\dim H^1$ per protocol state under a simplified four-channel model.

State	Upper-bound $\dim H^1$	**Key asymmetry**
Initiated	6	Off-chain task and budget negotiation remain bilateral
Quoted	7	Quote semantics and acceptance are not uniformly replicated
Committed	7	Escrow linkage semantics and timing are not fully shared as meaning
InProgress	7	Progress semantics remain partly off-chain
Delivered	8	Delivery evidence and metadata are split across channels
Settled	7	Public settlement references do not exhaust settlement semantics
Disputed	8	Dispute rationale and evidence remain asymmetric
Cancelled	6	Cancellation reason and refund semantics are not uniformly visible

Proposition 5.4. *Under the simplified four-channel model, ACTP v2.7 satisfies*

$$\dim H^1(\mathcal{F}^s_{P,\mathrm{sem}}) \geq 6$$

for every ACTP state s.

Proof. By direct computation in the supplementary script `participant_semantic_h1.py` on the simplified four-channel model. Because this abstraction compresses public chain visibility and leaves the richest semantic payloads on requester/provider channels, the result is a semantic upper-bound, not an exact implementation-level visibility invariant. □

The supplementary script `participant_semantic_h1.py` reproduces the Table 4 values exactly for the conservative semantic four-channel model. Separately, `participant_h1.py` implements the exact six-channel raw-visibility participant complex using the six edges of K_4 and four explicit public-information faces. In that higher-order model, every ACTP state yields $H^1 = 0$; see Appendix E. This does *not* refute the upper-bound table above. Instead it shows that raw visibility and semantic symmetry are different sheaf-theoretic questions: once public cycle classes are explicitly filled, the remaining asymmetry is no longer captured by simple visibility intersections alone.

The two *peak* states in this upper-bound model are DELIVERED and DISPUTED with $H^1 = 8$.

Delivered: pre-dispute window. At delivery, all principals can inspect the public on-chain transaction record, but they do not all share the same *semantic* view of delivery. Requester and Provider may hold richer delivery evidence or content hashes off-chain, while EscrowVault and ACTPKernel primarily expose settlement and state references. The asymmetry is therefore between semantic evidence and public references, not between public and private chain state.

Disputed: dispute initiation. The existence of a dispute is public, but the structured rationale and the supporting evidence package are not necessarily replicated as rich semantic payloads to all principals. Provider can observe that a dispute exists; what remains asymmetric is the full evidentiary context required to resolve it without privileged intervention.

5.3 Implications for Trustlessness

Corollary 5.5. *Structural completeness (*$\dim H^1(\mathcal{F}_S) = 0$ *on the 2-complex) is necessary but not sufficient for trustlessness. Positive participant cohomology in the conservative semantic model,* $\dim H^1(\mathcal{F}^s_{P,\mathrm{sem}}) > 0$*, measures residual coordination burden at the semantic resolution of that model.*

The three protocol tests — Trustless, Walkaway, Self-Sovereign[1] — admit a precise algebraic interpretation in terms of the participant H^1:

- **Trustless:** $H^1(\mathcal{F}_S) = 0$ (structural) $\wedge$ $H^1(\mathcal{F}^{\mathrm{sem}}_P) = 0$ (participant). The conservative semantic participant model fails this test for ACTP, while the exact higher-order raw-visibility supplement asks a strictly weaker question and collapses to zero after public-face completion.
- **Walkaway:** Each principal can reconstruct their obligations from on-chain data alone — a property related to, but not identical with, $H^1(\mathcal{F}^{\mathrm{sem}}_P) = 0$.
- **Self-Sovereign:** No single party controls the outcome — captured by the absence of any stalk dimension reachable only through a single participant's channel.

[1] Originally proposed as informal design criteria for ACTP [1].

Model-dependent verdict. ACTP satisfies the Trustless criterion under the exact six-channel raw-visibility model ($H^1(\mathcal{F}^s_{P,\text{raw}}) = 0$ at every state) but not under the conservative semantic model ($H^1(\mathcal{F}^s_{P,\text{sem}}) \geq 6$). This gap formalises the distinction between what participants can *observe* (public on-chain references) and what they can *semantically verify* (full evidentiary content). Whether the protocol is "trustless" therefore depends on the modelling resolution, and that dependency is itself part of the framework's contribution.

Remark 5.6. Reducing $\dim H^1(\mathcal{F}_{P,\text{sem}})$ is a concrete protocol-design objective: one can in principle enumerate the obstruction generators and design new information-sharing mechanisms (e.g., on-chain dispute reason commitment, delivery verification oracle) that close each gap.

6 Tooling

All supplementary code and protocol definitions are available at github.com/agirails/actp-sheaf-cohomology. The repository provides three Python modules: `h1_engine.py`, `h1_lint.py`, and the older `actp_sheaf.py` reference script. The first two constitute the current public artifact and reproduce both the graph-based / 1-complex diagnostics and the explicit face-based 2-complex analysis used in this paper's implementation-grounded results.

6.1 Protocol Specification Format

Protocols are specified in YAML with a deliberately small schema:

```yaml
states:
  INITIATED:
    stalks:
      - transaction_id
      - requester_address
      - provider_address
      - amount
      - deadline
      - service_hash
  QUOTED:
    stalks:
      - transaction_id
      - requester_address
      - provider_address
      - amount
      - deadline
      - service_hash
      - quote_details
transitions:
  - { from: INITIATED, to: QUOTED, name: submitQuote }
  - { from: INITIATED, to: COMMITTED, name: linkEscrow_direct }
paths:
  happy:
    states: [INITIATED, COMMITTED, IN_PROGRESS, DELIVERED, SETTLED]
    expect_h1: 0
faces:
  - name: quote_commit
    plus: [submitQuote, linkEscrow_quoted]
```

```
    minus: [linkEscrow_direct]
```

Listing 1: Excerpt of `protocol.yaml` for ACTP v2.7.

6.2 h1_engine.py: Core Computation

The current public engine performs both the graph-based / 1-complex and the explicit face-based 2-complex computations:

1. parse state stalks and transition endpoints from YAML;
2. derive each edge stalk as the field intersection of its endpoints;
3. when `faces:` are declared, derive each face stalk as the ordered intersection of the edge stalks on its boundary;
4. assemble $\delta_0 \in \mathbb{R}^{n_e \times n_v}$ and $\delta_1 \in \mathbb{R}^{n_f \times n_e}$;
5. compute the 1-complex and 2-complex cohomology dimensions.

6.3 h1_lint.py: CI Integration

The linter wraps the current engine with:

- **Regression testing:** `expect_h1` inside named paths triggers assertion; non-zero exit code on failure enables CI gating.
- **Diff mode:** `h1_lint.py --diff v2.yaml v3.yaml` reports $\Delta \dim H^1$ per subgraph, highlighting regressions.
- **JSON output:** `--json` flag produces machine-readable output for dashboard integration.
- **Leak tracing:** `--leaks` and `--path` expose which dimensions are lost or gained at each transition.
- **2-complex mode:** `--2complex` computes $\operatorname{rank}(\delta_1)$, $\dim \ker(\delta_1)$, $\dim H^1$, $\dim H^2$, and reports the numerical residual of $\delta_1 \circ \delta_0$.

Participant supplements. The public artifact also includes two participant-specific supplements. `participant_semantic_h1.py` reproduces the conservative four-channel semantic participant table from Section 5, while `participant_h1.py` computes the exact six-channel raw-visibility participant complex and its higher-order face completion. Together they make the paper's semantic/raw-visibility distinction reproducible rather than purely expository.

Sample output:

```
$ python3 h1_lint.py --config protocol.yaml --2complex
H^1 PROTOCOL LINTER - ACTP v2.7.0
...
FULL PROTOCOL (1-COMPLEX)
  H^1 =  24  24 obstructions
...
FULL PROTOCOL (2-COMPLEX)
  H^1 =   0  topologically complete
  H^2 =  20
```

```
  C^0 =  66  C^1 =  72  C^2 =  44
  rank(delta_0) = 48  rank(delta_1) = 24  dim ker(delta_1) = 48
...
PATHS
  happy              H^1 = 0   - topologically complete
  happy_quoted       H^1 = 6   - Quote-specific data is not fully carried
     forward in the 1-complex
  dispute            H^1 = 6   - 6 obstructions
  cancellation       H^1 = 18  - Terminal state - obstructions acceptable
     by design
```

Listing 2: Current public linter output for ACTP v2.7. The path name `happy_quoted` corresponds to the Quote/commit comparison subgraph in Table 2.

7 Related Work

Sheaves for distributed systems. Felber et al. [6] construct task sheaves over distributed systems and show that sheaf cohomology encodes obstructions to task solvability: global sections of the task sheaf correspond to valid protocol solutions. This paper shares the sheaf-cohomological methodology but applies it to settlement protocol verification rather than general distributed task solvability, and adds the participant information-asymmetry sheaf as a second construction.

Robinson [11] develops sheaf theory for distributed sensor fusion and signal processing, proving that H^1 counts the independent measurement inconsistencies across a network. Our restriction maps are analogous to his inter-sensor overlap maps.

Applied algebraic topology. Ghrist [7] provides the pedagogical foundation for cellular sheaves used throughout this paper, including the Scythe algorithm for computing persistent cohomology. Curry [4] gives the category- theoretic treatment; Hansen and Ghrist [8] derive spectral theory and connection Laplacians for cellular sheaves, which we do not exploit here but which could yield spectral obstruction measures in future work.

Formal methods for protocols. TLA$^+$ [10] and Alloy [9] are the standard tools for state-machine protocol verification; they check temporal and relational properties via model checking. Sheaf cohomology is complementary to these tools: it is not a model checker and does not enumerate states, but rather a topological invariant that measures *structural* consistency. Neither TLA$^+$ nor Alloy can directly compute information asymmetry as a group-theoretic quantity.

Smart-contract security. Formal verification of smart contracts (e.g., Certora [3], Echidna [5], Slither [12]) focuses on code-level properties (reentrancy, overflow, access control). ACTP's structural completeness sits one level above: it concerns the protocol state machine, not the Solidity implementation. Both layers are necessary.

8 Discussion

Two orthogonal concerns. Structural completeness and information symmetry are independent properties. A protocol can be structurally complete ($H^1(\mathcal{F}_S) = 0$) yet profoundly asym-

metric ($H^1(\mathcal{F}_P^{\text{sem}}) \gg 0$), as ACTP demonstrates. Conversely, a protocol could be symmetric but structurally incomplete (a hypothetical fully-transparent protocol with missing transitions). Both sheaves are necessary for a complete verification picture.

1-complex vs. 2-complex. When a higher-order face model is explicitly constructed, practitioners should use the 2-complex result as the ground truth for topological completeness, and the 1-complex result as a design signal. A positive 1-complex H^1 that collapses to zero on the 2-complex is not an error but a diagnostic: it identifies the specific transition stalks where data is shared implicitly via parallel paths rather than explicitly through matched shared observables. In practice, we recommend making the observable basis and transition surfaces explicit, and using the 1-complex linter as a first-pass regression check.

Practical protocol design. The linter enables:

1. **Regression gating.** Any protocol change that raises H^1 on the 1-complex or breaks a named path expectation triggers a review of the new stalk design.
2. **Version diff.** The diff mode localises regressions to specific transitions, accelerating root-cause analysis.
3. **Participant audit.** Computing $\dim H^1(\mathcal{F}^s_{P,\text{sem}})$ for a conservative participant model before deployment can guide oracle architecture decisions.

Generality beyond ACTP. To confirm that the methodology is not ACTP-specific, we applied the same tooling to two minimal escrow protocols. A 3-state linear lifecycle (Lock $\to$ Deliver $\to$ Release) with full field carry-through yields $H^1 = 0$ on the 1-complex, as expected for an acyclic graph. Adding a dispute branch (Deliver $\to$ Dispute $\to$ Release) that drops the `amount` field creates a cycle with an information mismatch; the linter immediately reports $H^1 = 4$, confirming that the tool detects structural gaps in arbitrary state-machine protocols. YAML definitions for both toy protocols are included in the supplementary repository.

Computational cost. On the current ACTP artifact, δ_0 is 72×66 and its rank is computed in milliseconds. Runtime is dominated by dense rank computations (`matrix_rank`); worst-case complexity is superlinear in matrix size. For the current artifact and moderate protocol sizes, this remains tractable on commodity hardware, and the linter can run on pull requests without special optimisation.

Raw visibility vs. semantic symmetry. The new supplementary participant script makes an important distinction explicit. On the exact six-channel raw-visibility participant complex, once the public-information triangles are added, participant H^1 collapses to zero in every ACTP state. This does not mean ACTP is fully trustless. Rather, it means that raw public observability alone is too coarse to encode the semantic gap between a public commitment (hash, UID, event) and the richer off-chain content it commits to. The conservative four-channel participant table in Section 5 should therefore be read as an upper-bound semantic diagnostic, not as the exact raw-visibility invariant.

Empirical mainnet execution. The analysis in this paper is not purely hypothetical. An ACTP transaction was executed on Base mainnet and completed an end-to-end settlement flow [2]. Local ABI-decode analysis of that transaction maps it to the implemented happy-path settlement interpretation: funds were escrowed, delivery was completed, and settlement was released on-chain without requiring manual custody intervention. This empirical case does *not* by itself prove the global 2-complex theorem, but it does show that the protocol has been exercised in practice on a production network and that the happy-path interpretation is not merely notional. Appendix C records the minimal explorer-visible signals used for that empirical claim.

Limitations. Several limitations are worth acknowledging:

1. **Static analysis only.** The sheaf is constructed from the protocol specification, not from runtime behaviour. Race conditions, re-entrancy, and network-partition effects are not modelled.
2. **No concurrency.** ACTP as modelled is a single-transaction lifecycle. A product sheaf over multiple concurrent transactions would be needed to reason about global settlement fund flows.
3. **Real-valued stalks.** We use $\mathbb{R}$-vector spaces for algebraic convenience. The actual data types (strings, hashes, addresses) are not typed; the dimension count is a proxy for information content, not an exact measure.
4. **Participant model abstraction.** Section 5 deliberately uses a simplified four-channel model to obtain a conservative semantic upper bound, now reproduced by `participant_semantic_h1.py`. The supplementary higher-order six-channel raw-visibility model now makes that exact question reproducible and yields $H^1 = 0$ after public-face completion. The remaining open problem is therefore not six-channel computation by itself, but refining the semantic participant sheaf so that its restriction maps distinguish content from public commitments in a less conservative, more implementation-faithful way.
5. **Calldata opacity.** The participant model treats transition proof payloads as opaque unless they are stored or surfaced through standard views and events. In practice, calldata is recoverable from block data, which could reduce participant asymmetry in states where proof content is material.
6. **Explicit face specification.** The current public 2-complex result depends on a declared `faces:` section in the YAML artifact. The supplement does not yet infer a canonical face decomposition automatically from the graph.
7. **Attestation enforcement.** The current kernel stores an `attestationUID` but does not validate EAS semantics on-chain; attestation verification remains optional at SDK level.

Future work.

- **Product sheaves for concurrent transactions.** Extend the framework to multi-agent settlement pools, where independent lifecycles share an escrow vault.
- **Refined semantic participant sheaf.** Extend the current reproducible conservative semantic supplement so that its restriction maps explicitly relate semantic content to

public commitments (hashes, UIDs, events), reducing reliance on deliberate upper-bound choices.

- **Runtime H^1 monitoring.** Instrument the ACTP SDK to compute sheaf consistency checks on live state transitions, raising alerts when runtime data deviates from the protocol specification.
- **Automatic face discovery.** Replace the current explicit `faces:` declarations with a verified face-generation pass so that the 2-complex decomposition is derived rather than hand-listed.
- **Spectral methods.** The Hodge Laplacian $L_1 = \delta_0\delta_0^\top + \delta_1^\top\delta_1$ provides a spectral measure of obstruction severity [8]; smallest non-zero eigenvalue may serve as a continuous completeness score.

9 Conclusion

Non-deterministic agents cannot safely transact without a provably correct escrow layer, and "provably correct" is not a single thing. This paper addressed one specific part of that question: whether the local state an agent settlement protocol exposes at each step assembles into a globally consistent picture, and where it fails to. Cellular sheaf cohomology provides a direct, computable answer. Applied to ACTP, the answer has two components:

1. **State-based sheaf.** On the current reduced-basis supplementary artifact derived from the implemented protocol, ACTP v2.7 yields $H^1 = 24$ on the 1-complex. In the current public 2-complex artifact, using one explicit 10-face refinement, these apparent obstructions collapse to $H^1 = 0$, separating edge-level design signals from topological completeness.
2. **Participant-based sheaf.** Under a conservative simplified four-channel semantic model, participant H^1 remains strictly positive and lies in $[6, 8]$ across all ACTP states, now reproduced exactly by the supplementary script `participant_semantic_h1.py`. Structural completeness therefore does not imply participant symmetry or trustlessness. A separate higher-order six-channel raw-visibility model yields $H^1 = 0$ after public-face completion, showing that the conservative participant table is a semantic upper-bound diagnostic rather than an exact raw-visibility invariant.
3. **Dispute and attestation analysis.** The implemented dispute process, not a separate mediation state, is the relevant trust surface for ACTP. In a paper-level 1-complex augmentation experiment, attestation observables without corresponding shared structure increase the obstruction count ($24 \to 26$). Current attestation verification remains SDK-side rather than kernel-enforced.
4. **Open-source tooling.** The current public artifact accepts YAML protocol definitions, computes graph-based / 1-complex H^1, computes explicit face-based 2-complex H^1 and H^2 via `--2complex`, includes `participant_semantic_h1.py` for the conservative semantic participant supplement and `participant_h1.py` for the exact six-channel raw-visibility supplement, performs version diffs, surfaces leak diagnostics, and emits JSON for CI integration.
5. **Empirical validation.** Beyond code inspection, ACTP has been exercised on Base mainnet in a recorded end-to-end settlement transaction [2]. That transaction is best read

as empirical support for implementation reality and happy-path execution, not as a standalone proof of the paper's higher-order cohomological claims.

To our knowledge, this is the first application of sheaf cohomology to settlement protocol verification. Structural completeness and participant symmetry are only two of the properties a full notion of trustlessness would require — formal definitions of strategic behaviour, dynamic analysis, and a bridge to the Solidity implementation all remain open. But making one of those properties rigorously measurable, reproducible, and CI-integrable is a starting point. We invite the protocol design community to treat H^1 linting as a standard component of protocol development, alongside model checking and smart-contract auditing.

Acknowledgements. We thank Björn Lenz, PhD (Mathematics, RWTH Aachen University) for independent mathematical review of the sheaf-theoretic constructions. Early exploration and computational prototyping were assisted by AI research agents (Anthropic Claude, OpenAI Codex); all mathematical claims were verified by the human authors.

References

[1] AGIRAILS Inc. AGIRAILS SDK Documentation and ACTP Protocol Specification. 2026. https://docs.agirails.io. Accessed 2026-04-06.

[2] BaseScan. Base mainnet transaction https://basescan.org/tx/0xaa98180f991cdaaf35b5e38c8f14c0d75bb9dd075061a13dfff48ec2b9ccff19. Accessed 2026-04-06.

[3] Certora. Certora Prover Documentation. https://docs.certora.com/. Accessed 2026-04-10.

[4] J. Curry. *Sheaves, Cosheaves and Applications*. PhD thesis, University of Pennsylvania, 2014. arXiv:1303.3255.

[5] Trail of Bits. Echidna: Ethereum smart contract fuzzer. https://github.com/crytic/echidna. Accessed 2026-04-10.

[6] S. Felber, B. Hummes Flores, and H. Rincon Galeana. A sheaf-theoretic characterization of tasks in distributed systems. arXiv:2503.02556 [cs.DC], 2025.

[7] R. Ghrist. *Elementary Applied Topology*. Createspace, 2014. https://www2.math.upenn.edu/~ghrist/EAT/EATchapter6.pdf.

[8] J. Hansen and R. Ghrist. Toward a spectral theory of cellular sheaves. *Journal of Applied and Computational Topology*, 3(4):315–358, 2019.

[9] D. Jackson. *Software Abstractions: Logic, Language, and Analysis*. MIT Press, revised edition, 2012.

[10] L. Lamport. *Specifying Systems: The TLA$^+$ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[11] M. Robinson. *Topological Signal Processing*. Springer, 2014.

[12] J. Feist, G. Grieco, and A. Groce. Slither: A static analysis framework for smart contracts. arXiv:1908.09878 [cs.CR], 2019.

A ACTP v2.7 Transition Table

Table 5: Supplementary-artifact transition list for ACTP v2.7 with shared-field counts.

ID	Source	Target	Carried fields (count)	$s(e)$
e_1	Initiated	Quoted	transaction_id, requester_address, provider_address, amount, deadline, service_hash	6
e_2	Initiated	Committed	transaction_id, requester_address, provider_address, amount, deadline, service_hash	6
e_3	Initiated	Cancelled	transaction_id, requester_address, provider_address, amount	4
e_4	Quoted	Committed	transaction_id, requester_address, provider_address, amount, deadline, service_hash	6
e_5	Quoted	Cancelled	transaction_id, requester_address, provider_address, amount	4
e_6	Committed	InProgress	transaction_id, requester_address, provider_address, amount, deadline, service_hash, escrow_id, escrow_balance	8
e_7	Committed	Cancelled	transaction_id, requester_address, provider_address, amount	4
e_8	InProgress	Delivered	transaction_id, requester_address, provider_address, amount, deadline, service_hash, escrow_id, escrow_balance, work_started_at	9
e_9	InProgress	Cancelled	transaction_id, requester_address, provider_address, amount	4
e_{10}	Delivered	Settled	transaction_id, requester_address, provider_address, amount, service_hash, escrow_id	6
e_{11}	Delivered	Disputed	transaction_id, requester_address, provider_address, amount, deadline, service_hash, escrow_id, escrow_balance, delivery_proof	9
e_{12}	Disputed	Settled	transaction_id, requester_address, provider_address, amount, service_hash, escrow_id	6
			Total $\dim C^1$	**72**

B Public 2-Cell Decomposition

The current public `protocol.yaml` declares the following 10 faces. Each face is a formal difference of two directed paths with identical endpoints, and its stalk dimension is the intersection of the edge stalks on that boundary.

Table 6: Explicit 2-cells in the public ACTP supplementary artifact.

Name	p^+	p^-	$\dim(f)$
`quote_commit`	e_1, e_4	e_2	6
`dispute_settle`	e_{11}, e_{12}	e_{10}	6
`cancel_via_quote`	e_1, e_5	e_3	4
`cancel_via_commit_direct`	e_2, e_7	e_3	4
`cancel_via_commit_quoted`	e_1, e_4, e_7	e_3	4
`cancel_via_inprog_direct`	e_2, e_6, e_9	e_3	4
`cancel_via_inprog_quoted`	e_1, e_4, e_6, e_9	e_3	4
`cancel_quote_vs_commit`	e_4, e_7	e_5	4
`cancel_quote_vs_inprog`	e_4, e_6, e_9	e_5	4
`cancel_commit_vs_inprog`	e_6, e_9	e_7	4
	Total $\dim C^2$		**44**

C Mainnet Execution Trace

Table 8 records the minimal explorer-visible evidence used for the empirical mainnet claim in Section 8. The table is intentionally modest: it is not presented as a proof of the global cohomology theorems, only as concrete evidence that the ACTP happy-path settlement interpretation has occurred on Base mainnet in a live transaction. Table 7 records the role mapping used in the local ABI decode of that transaction.

Table 7: Role mapping used for the Base mainnet ACTP settlement trace.

ACTP role	Base address	Interpretation
Requester	`0xf71eB519...Ff3DF0ac2`	AI-agent smart account initiating the settlement flow
Provider	`0x9ee3a01a...f61292d20`	AI-agent counterparty receiving escrow release
ACTPKernel	`0x132B9eB3...1BDC92d29`	Protocol state-machine contract
EscrowVault	`0x6aAF4588...6be7fFb99`	Non-custodial USDC escrow contract
Paymaster	`0x2FAEB076...D634FD4c`	ERC-4337 gas sponsor for the user operation

Table 8: Explorer-visible signals for the Base mainnet ACTP settlement transaction [2].

Signal	Explorer-visible observation	ACTP interpretation
Transaction metadata	Base mainnet transaction dated 2026-02-21 for hash `0xaa9818...cff19`	Production-network execution rather than a local or simulated trace
State transition log	Explorer event stream shows a state change from code 4 to code 5 for the same ACTP escrow context (local ABI decode: log 466)	Implemented DELIVERED→ SETTLEDtransition on the happy path
Payout transfer	Explorer-visible token transfer sends 3.6531 USDC from escrow custody to the Provider address (local ABI decode: log 467)	Non-custodial escrow release to the service provider
Settlement event	Final release event records provider payout and platform fee (local ABI decode: log 469)	Settlement bookkeeping completed on-chain

D Participant Semantic Supplement

The supplementary script `participant_semantic_h1.py` reproduces the conservative four-channel semantic participant model used in Section 5. Table 9 records the resulting cochain dimensions and ranks for each ACTP state.

E Participant Higher-Order Supplement

The supplementary script `participant_h1.py` models the exact six-channel participant raw-visibility surface on the 2-skeleton of K_4: the four principals are vertices, the six information channels are edges, and the four public-information triangles are explicit 2-cells. Under this exact raw-visibility model, graph-only cycle classes disappear once the public faces are filled.

Read this table alongside Section 5. The large graph-only six-channel counts are topological artefacts of leaving public visibility cycles unfilled, while the zero higher-order counts show that raw visibility alone is too coarse to witness the semantic asymmetry emphasised in the main text.

Table 9: Exact outputs of `participant_semantic_h1.py` for the conservative semantic participant model.

State	$\dim C^0$	$\dim C^1$	$\operatorname{rank}(\delta_0)$	$\dim H^1$
Initiated	30	22	16	6
Quoted	35	26	19	7
Committed	38	27	20	7
InProgress	40	28	21	7
Delivered	43	31	23	8
Settled	38	27	20	7
Disputed	43	30	22	8
Cancelled	31	22	16	6

Table 10: Exact six-channel participant raw-visibility results from `participant_h1.py`.

State	Graph-only six-channel $\dim H^1$	Higher-order six-channel $\dim H^1$
Initiated	24	0
Quoted	27	0
Committed	39	0
InProgress	42	0
Delivered	45	0
Settled	39	0
Disputed	48	0
Cancelled	21	0

F Euler Characteristic Cross-Check

By the Euler–Poincaré formula for sheaf cohomology:

$$\chi(\mathcal{F}) = \dim H^0 - \dim H^1 + \dim H^2 = \dim C^0 - \dim C^1 + \dim C^2.$$

For ACTP v2.7 on the 2-complex:

$$\chi = 66 - 72 + 44 = 38.$$

Using cohomology dimensions: $\chi = 18 - 0 + 20 = 38$. ✓